

实验二、三实验报告

实验二 数据库表的创建与维护实验

实验目的

- 1. 通过进行数据库表的建立操作，熟悉并掌握GaussDB数据库表的建立方法，理解关系型数据库表的结构，巩固PostgreSQL中关于数据库表的建立语句；
- 2. 通过进行数据库表数据的增加、删除和插入等维护操作，熟悉并掌握GaussDB数据库数据的操作方法，巩固PostgreSQL中关于数据维护的语句。

实验平台及环境

- 操作系统：Windows 11
- 平台：GaussDB
- 使用语言：PostgreSQL

实验内容

- 1. 在GaussDB数据库中创建相应的数据表；
- 2. 对表进行数据导入、增加、删除、修改、查询等基本操作；
- 3. 掌握表结构的修改操作（增加列、修改列数据类型、删除列）；
- 4. 学习数据表的删除操作。

实验步骤及结果

- 1. 修改“用户基本信息”表数据，增加名为“备注”的列,数据类型为varchar()型；

SQL: ALTER TABLE 用户基本信息表 ADD COLUMN remark VARCHAR(200)

user_id	username	password	email	phone	age	gender	create_time	update_time	remark
1	张三	123456	zhangsan@gaussdb.com	13800000000	28	男	2023-01-01 10:00:00	2023-01-01 10:00:00	(null)
2	李四	654321	lisi@gaussdb.com	13900000000	24	男	2023-01-02 11:00:00	2023-01-02 11:00:00	(null)
3	王五	987654	wangwu@gaussdb.com	13700000000	32	男	2023-01-03 09:00:00	2023-01-03 09:00:00	(null)
4	赵六	234567	zhao6@gaussdb.com	13600000000	26	男	2023-01-04 15:00:00	2023-01-04 15:00:00	(null)
5	孙七	789012	sun7@gaussdb.com	13500000000	21	男	2023-01-05 12:00:00	2023-01-05 12:00:00	(null)
6	周八	345678	zhou8@gaussdb.com	13400000000	45	男	2023-01-06 10:00:00	2023-01-06 10:00:00	(null)
7	吴九	890123	wu9@gaussdb.com	13300000000	29	男	2023-01-07 13:00:00	2023-01-07 13:00:00	(null)
8	郑十	456789	zheng10@gaussdb.com	13200000000	27	男	2023-01-08 11:00:00	2023-01-08 11:00:00	(null)
9	赵十一	123456	zhao11@gaussdb.com	13100000000	33	男	2023-01-09 14:00:00	2023-01-09 14:00:00	(null)
10	孙十二	654321	sun12@gaussdb.com	13000000000	25	男	2023-01-10 16:00:00	2023-01-10 16:00:00	(null)
11	周十三	987654	zhou13@gaussdb.com	12900000000	23	男	2023-01-11 08:00:00	2023-01-11 08:00:00	(null)
12	吴十四	234567	wu14@gaussdb.com	12800000000	20	男	2023-01-12 17:00:00	2023-01-12 17:00:00	(null)
13	郑十五	789012	zheng15@gaussdb.com	12700000000	22	男	2023-01-13 10:00:00	2023-01-13 10:00:00	(null)
14	赵十六	345678	zhao16@gaussdb.com	12600000000	24	男	2023-01-14 19:00:00	2023-01-14 19:00:00	(null)
15	孙十七	890123	sun17@gaussdb.com	12500000000	26	男	2023-01-15 07:00:00	2023-01-15 07:00:00	(null)
16	周十八	456789	zhou18@gaussdb.com	12400000000	28	男	2023-01-16 20:00:00	2023-01-16 20:00:00	(null)

- 2. 修改“用户基本信息”表数据，将“备注”列的数据类型改为int；

SQL: ALTER TABLE 用户基本信息表 ALTER COLUMN remark TYPE INT USING (remark::integer);

user_id	username	password	email	phone	age	gender	create_time	update_time	remark
1	张三	zhangsan123	zhangsan@example.com	13000130001	26	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
2	李四	lisi456	lisi@example.com	13000130002	24	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
3	王五	wangwu789	wangwu@example.com	13000130003	22	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
4	赵六	zhao67890	zhao6@example.com	13000130004	20	女	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
5	钱七	qianq78901	qianq@example.com	13000130005	25	女	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
6	孙八	sunba89012	sunba@example.com	13000130006	40	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
7	黄九	huang90123	huang9@example.com	13000130007	18	女	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
8	郑十	zheng101234	zheng10@example.com	13000130008	27	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
9	王五	wang1012345	wang10@example.com	13000130009	21	女	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
10	刘强	liuqc1234567	liuqc@example.com	13000130010	29	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
11	李强	liuqc1234567	liuqc@example.com	13000130011	25	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
12	陈静	chenj12345678	chenj@example.com	13000130012	26	女	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
13	陈强	chengq12345678	chengq@example.com	13000130013	31	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
14	高飞	gaofei123456789	gaofei@example.com	13000130014	24	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
15	张强	zhangq1234567890	zhangq@example.com	13000130015	29	女	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
16	郭强	guoq12345678901	guoq@example.com	13000130016	35	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
17	成强	chengq123456789012	chengq@example.com	13000130017	23	女	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)
18	马强	maqc1234567890123	maqc@example.com	13000130018	32	男	2023-01-01 00:00:00	2023-01-01 00:00:00	(remark)

3. 修改“用户基本信息”表数据，删除“备注”列

SQL: ALTER TABLE 用户基本信息表 DROP COLUMN remark

user_id	username	password	email	phone	age	gender	create_time	update_time	
1	张三	zhangsan123	wangq@example.com	13000130001	26	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
2	李四	lisi456	wangq@example.com	13000130002	24	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
3	王五	wangwu789	wangq@example.com	13000130003	22	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
4	赵六	zhao67890	zhao6@example.com	13000130004	20	女	2023-01-01 00:00:00	2023-01-01 00:00:00	
5	钱七	qianq78901	qianq@example.com	13000130005	25	女	2023-01-01 00:00:00	2023-01-01 00:00:00	
6	孙八	sunba89012	sunba@example.com	13000130006	40	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
7	黄九	huang90123	huang9@example.com	13000130007	18	女	2023-01-01 00:00:00	2023-01-01 00:00:00	
8	郑十	zheng101234	zheng10@example.com	13000130008	27	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
9	王五	wang1012345	wang10@example.com	13000130009	21	女	2023-01-01 00:00:00	2023-01-01 00:00:00	
10	刘强	liuqc1234567	liuqc@example.com	13000130010	29	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
11	李强	liuqc1234567	liuqc@example.com	13000130011	25	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
12	陈静	chenj12345678	chenj@example.com	13000130012	26	女	2023-01-01 00:00:00	2023-01-01 00:00:00	
13	陈强	chengq12345678	chengq@example.com	13000130013	31	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
14	高飞	gaofei123456789	gaofei@example.com	13000130014	24	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
15	张强	zhangq1234567890	zhangq@example.com	13000130015	29	女	2023-01-01 00:00:00	2023-01-01 00:00:00	
16	郭强	guoq12345678901	guoq@example.com	13000130016	35	男	2023-01-01 00:00:00	2023-01-01 00:00:00	
17	成强	chengq123456789012	chengq@example.com	13000130017	23	女	2023-01-01 00:00:00	2023-01-01 00:00:00	
18	马强	maqc1234567890123	maqc@example.com	13000130018	32	男	2023-01-01 00:00:00	2023-01-01 00:00:00	

4. 删除“用户基本信息”数据表

SQL:

DROP TABLE IF EXISTS "购物车基本信息表";

DROP TABLE IF EXISTS "地址基本信息表";

DROP TABLE IF EXISTS "订单明细基本信息表";

DROP TABLE IF EXISTS "订单基本信息表";

DROP TABLE IF EXISTS "用户基本信息表";

实验小结

在实验中，在我想要执行删除“用户基本信息表”任务DROP TABLE IF EXISTS "用户基本信息表";时，会出现报错。主要原因是：外键约束依赖，其他表的外键约束了用户基本信息表，导致无法直接删除。此外，其他的表页会被外键约束。以下表都引用了"用户基本信息表"的 user_id 字段：

1. 地址基本信息表 - 外键：fk_address_user
2. 订单基本信息表 - 外键：fk_order_user
3. 购物车基本信息表 - 外键：fk_cart_user

解决方案：可以按顺序将所有有依赖关系的表全部删除，或者使用级联删除。

实验三 数据查询实验

实验目的

通过对实验二建立的数据库关系表的各种查询的操作，加深对SQL语言和PostgreSQL查询语言的了解，掌握相关查询语句的语法及使用方法。

实验平台及环境

- 操作系统：Windows 11
- 平台：GaussDB
- 使用语言：PostgreSQL

实验内容

1.单表查询

- 1-1 查询所有订单的创建日期。
- 1-2 查询每个订单明细的商品总价。
- 1-3 查询订单基本信息表中2023年3月的所有订单。
- 1-4 查询用户基本信息表中用户名以"张"或"李"开头的用户。
- 1-5 查询全部的订单信息，先按照用户编号升序排序，再按照订单总金额降序排序
- 1-6 统计每个用户的订单数量，最大订单金额，最小订单金额，平均订单金额以及总订单金额。
- 1-7 查找购物车中商品数量超过5的用户。
- 1-8 统计每个年龄段的人数（人群按照年龄划分，未成年<18,18<=青壮年<60, 老年>=60）
- 1-9 获取价格最高的前5个商品。

2.多表查询与嵌套查询：

- 2-1 根据用户基本信息表和订单基本信息表，查询所有用户的所有订单信息。
- 2-2 根据商品分类基本信息表查询每个分类的父分类名称
- 2-3 查询所有用户和购物车记录的完整信息（包括查询没有购物车信息的用户）。
- 2-4 查询所有包含商品编号为10的订单信息。
- 2-5 查询订单总金额大于平均订单金额的所有订单信息。
- 2-6 查询订单总金额大于任意一个用户的最小订单金额的订单信息。
- 2-7 查询订单总金额大于所有用户的平均订单金额的所有订单信息。
- 2-8 使用EXISTS谓词查询至少购买过一次商品的个人信息。
- 2-9 查询每个商品分类的名称及其总销售额。

3.集合查询：

- 3-1 查询所有购买了商品10或商品20的订单（包括重复订单）。
- 3-2 查询所有同时购买了商品10和商品20的订单。
- 3-3 查询购买了商品10但没有购买商品20的订单

实验步骤及结果

1. 单表查询

1-1 查询所有订单的创建日期。

SELECT create_time FROM 订单基本信息表;

	create_time
1	2023-09-04 00:00:00
2	2023-05-11 00:00:00
3	2023-06-19 00:00:00
4	2023-09-14 00:00:00
5	2023-01-13 00:00:00
6	2023-10-16 00:00:00
7	2023-05-29 00:00:00
8	2023-01-30 00:00:00
9	2023-04-08 00:00:00
10	2023-09-13 00:00:00
11	2023-12-23 00:00:00
12	2023-12-30 00:00:00

1-2 查询每个订单明细的商品总价。

SELECT
order_id,
SUM(price * quantity) AS sum_price
FROM
"订单明细基本信息表"
GROUP BY
order_id

	order_id	sum_price
1	1	20630.00
2	2	2799.00
3	3	3278.00
4	4	62294.00
5	5	2826.00
6	6	40082.00
7	7	843.00
8	8	45090.00
9	9	5397.00

1-3 查询订单基本信息表中2023年3月的所有订单。

SELECT *
FROM
"订单明细基本信息表"
WHERE
create_time >= '2023-03-01'
AND create_time < '2023-04-01'

	order_item_id	order_id	product_id	quantity	price	create_time
1	5	2	30	4	59.00	2023-03-11 00:00:00
2	7	2	27	5	69.00	2023-03-11 00:00:00
3	66	20	13	4	6999.00	2023-03-25 00:00:00
4	67	20	20	2	129.00	2023-03-26 00:00:00
5	68	20	17	2	2999.00	2023-03-24 00:00:00
6	117	34	30	2	59.00	2023-03-30 00:00:00
7	118	34	14	5	499.00	2023-03-30 00:00:00
8	119	34	6	2	349.00	2023-03-29 00:00:00
9	117	46	6	3	349.00	2023-03-05 00:00:00

1-4 查询用户基本信息表中用户名以"张"或"李"开头的用户。

```
SELECT * FROM "用户基本信息表"
WHERE username LIKE '张%' OR username LIKE '李%';
```

	user_id	username	password	email	phone	age	gender	create_time
1	1	张三	zhangsan123	zhangsan@example.com	13800138000	20	男	2023-01-15 00:00:00
2	2	李四	lisi1456	lisi@example.com	13800138001	34	男	2023-02-10 00:00:00
3	22	张丽	zhanglix567	zhangli@example.com	13800138011	26	女	2023-02-15 00:00:00
4	24	张娜	zhangna567	zhangna@example.com	13800138013	24	女	2023-04-05 00:00:00
5	101	张三21111	test123	test@example.com	13800138000	25	男	2025-02-09 00:00:00
6	107	张大爷	zhangsany123	zhangsany@example.com	13800138000	20	男	2023-01-15 00:00:00
7	108	张大爷2	zhangsany2123	zhangsany2@example.com	13800138000	20	男	2023-01-15 00:00:00

1-5 查询全部的订单信息，先按照用户编号升序排序，再按照订单总金额降序排序

```
SELECT
    user_id,
    total_amount
FROM
    "订单基本信息表"
ORDER BY
    user_id ASC,
    total_amount DESC;
```

	user_id	total_amount
1	1	84615.00
2	1	77167.00
3	1	67187.00
4	1	65615.00
5	1	62294.00
6	1	62186.00
7	1	61710.00
8	1	53994.00
9	1	48705.00

1-6 统计每个用户的订单数量，最大订单金额，最小订单金额，平均订单金额以及总订单金额。

```
SELECT
    user_id,
    COUNT(order_id),
    MAX(total_amount),
```

```

MIN(total_amount),
AVG(total_amount),
SUM(total_amount)

```

```
FROM
```

```
"订单基本信息表"
```

```
GROUP BY
```

```
user_id
```

```
ORDER BY
```

```
user_id ASC;
```

	user_id	count	max	min	avg	sum
1	1	188	84625.00	48.00	19999.000000000000	1999908.00
2	2	188	64335.00	115.00	17699.850000000000	1769905.00
3	3	188	89585.00	38.00	16929.500000000000	1692908.00
4	4	188	77285.00	156.00	17732.420000000000	1773242.00
5	5	188	78485.00	98.00	16273.240000000000	1627324.00
6	6	188	113384.00	114.00	28161.930000000000	2816193.00
7	7	188	113326.00	254.00	15829.370000000000	1582937.00
8	8	188	73788.00	138.00	16882.300000000000	1688238.00
9	9	188	75888.00	129.00	17166.890000000000	1716689.00

1-7 查找购物车中商品数量超过5的用户。

```
SELECT
```

```
user_id,
```

```
SUM(quantity) as sumq
```

```
FROM
```

```
购物车基本信息表
```

```
GROUP BY
```

```
user_id
```

```
HAVING
```

```
sumq>5
```

	user_id	sumq
1	15	9
2	42	17
3	91	11
4	56	11
5	29	14
6	63	13
7	33	18
8	1	10
9	68	18

1-8 统计每个年龄段的人数（人群按照年龄划分，未成年<18,18<=青壮年<60, 老年>=60）

```
SELECT
```

```
CASE
```

```

    WHEN age < 18 THEN '未成年(<18)'

    WHEN age >= 18 AND age < 60 THEN '青壮年(18-59)'

    WHEN age >= 60 THEN '老年(>=60)'

END AS 年龄段,

COUNT(*) AS 人数

FROM 用户基本信息表

GROUP BY

CASE

    WHEN age < 18 THEN '未成年(<18)'

    WHEN age >= 18 AND age < 60 THEN '青壮年(18-59)'

    WHEN age >= 60 THEN '老年(>=60)'

END

ORDER BY

CASE

    WHEN MIN(age) < 18 THEN 1

    WHEN MIN(age) >= 18 AND MIN(age) < 60 THEN 2

    WHEN MIN(age) >= 60 THEN 3

END;

```

	年龄段	人数
1	未成年(<18)	181
2	青壮年(18-59)	9

1-9 获取价格最高的前5个商品。

```

SELECT

    "name",

    price

FROM

    商品基本信息表

ORDER BY

    price DESC

LIMIT 5

```

	name	price
1	苹果 iPhone 14 Pro	8999.00
2	索尼 WH-1000XM4 无线降噪耳机	8999.00
3	苹果 iPad Pro	7999.00
4	小米 13 Pro 5G 智能手机	6999.00
5	华为 Mate 60 Pro 5G 智能手机	5999.00

2. 多表查询与嵌套查询：

2-1 根据用户基本信息表和订单基本信息表，查询所有用户的所有订单信息。

SELECT

```
u.user_id,  
u.username,  
u.email,  
o.order_id,  
o.total_amount,  
o.create_time AS order_create_time,  
o.update_time AS order_update_time
```

FROM "用户基本信息表" u

INNER JOIN "订单基本信息表" o ON u.user_id = o.user_id

ORDER BY u.user_id, o.create_time DESC;

	user_id	username	email	order_id	total_amount	order_create_time	order_update_time
1	1	张三	changsan@example.com	12	18582.00	2023-12-30 00:00:00	2024-01-01 00:00:00
2	1	张三	changsan@example.com	50	1813.00	2023-12-28 00:00:00	2023-12-30 00:00:00
3	1	张三	changsan@example.com	31	634.00	2023-12-25 00:00:00	2023-12-26 00:00:00
4	1	张三	changsan@example.com	53	62106.00	2023-12-20 00:00:00	2024-01-15 00:00:00
5	1	张三	changsan@example.com	81	3962.00	2023-12-19 00:00:00	2023-12-25 00:00:00
6	1	张三	changsan@example.com	93	20033.00	2023-12-11 00:00:00	2023-12-21 00:00:00
7	1	张三	changsan@example.com	69	6257.00	2023-12-05 00:00:00	2023-12-09 00:00:00
8	1	张三	changsan@example.com	25	48705.00	2023-12-04 00:00:00	2023-12-13 00:00:00
9	1	张三	changsan@example.com	77	1393.00	2023-11-23 00:00:00	2023-12-17 00:00:00

2-2 根据商品分类基本信息表查询每个分类的父分类名称

SELECT

```
c.category_id AS 分类ID,  
c.name AS 分类名称,  
p.name AS 父分类名称,  
c.create_time AS 创建时间
```

FROM "商品分类基本信息表" c

INNER JOIN "商品分类基本信息表" p ON c.parent_id = p.category_id

ORDER BY c.category_id;

	分类id	分类名称	父分类名称	创建时间
1	10	图书	商品	2025-09-13 11:13:00.908413
2	11	糕点	商品	2025-09-13 11:13:00.908413
3	12	药品	商品	2025-09-13 11:13:00.908413
4	13	鞋子	商品	2025-09-13 11:13:00.415678
5	14	运动鞋	鞋子	2025-09-13 11:13:00.908413
6	15	帽子	配饰	2025-09-13 11:13:00.908413
7	16	手机	电子产品	2025-09-13 11:13:00.908413
8	17	平板电脑	电子产品	2025-09-13 11:13:00.908413
9	18	笔记本电脑	电子产品	2025-09-13 11:13:00.908413

2-3 查询所有用户和购物车记录的完整信息（包括查询没有购物车信息的用户）。

```
SELECT
    u.user_id,
    u.username,
    u.email,
    u.phone,
    u.age,
    u.gender,
    u.create_time AS user_create_time,
    u.update_time AS user_update_time,
    c.cart_id,
    c.product_id,
    c.quantity AS cart_quantity,
    c.create_time AS cart_create_time,
    p.name AS product_name,
    p.price AS product_price,
    p.stock AS product_stock
FROM "用户基本信息表" u
LEFT JOIN "购物车基本信息表" c ON u.user_id = c.user_id
LEFT JOIN "商品基本信息表" p ON c.product_id = p.product_id
ORDER BY u.user_id, c.create_time DESC;
```

	user_id	username	email	phone	age	gender	user_create_time	user_update_time
1	1	张三	zhangsan@example.com	13800138000	28	男	2023-01-15 00:00:00	2023-10-01 00:00:00
2	1	张三	zhangsan@example.com	13800138000	28	男	2023-01-15 00:00:00	2023-10-01 00:00:00
3	2	李四	lisi@example.com	13800138001	34	男	2023-02-10 00:00:00	2023-10-02 00:00:00
4	3	王五	wangwu@example.com	13800138002	22	男	2023-03-05 00:00:00	2023-10-03 00:00:00
5	4	赵六	zhaoliu@example.com	13800138003	30	女	2023-04-20 00:00:00	2023-10-04 00:00:00
6	5	孙七	sunqi@example.com	13800138004	25	女	2023-05-12 00:00:00	2023-10-05 00:00:00
7	6	周八	zhouba@example.com	13800138005	40	男	2023-06-18 00:00:00	2023-10-06 00:00:00
8	7	吴九	wujiu@example.com	13800138006	33	女	2023-07-22 00:00:00	2023-10-07 00:00:00

2-4 查询所有包含商品编号为10的订单信息。

```

SELECT
    o.order_id,
    o.user_id,
    o.total_amount,
    o.create_time,
    o.update_time
FROM "订单基本信息表" o
INNER JOIN "订单明细基本信息表" oi ON o.order_id = oi.order_id
WHERE oi.product_id = 10

```

	order_id	user_id	total_amount	create_time	update_time
1	8	1	45898.00	2023-01-30 00:00:00	2023-02-16 00:00:00
2	15	1	45126.00	2023-07-23 00:00:00	2023-07-25 00:00:00
3	15	1	45126.00	2023-07-23 00:00:00	2023-07-25 00:00:00
4	21	1	18214.00	2023-06-25 00:00:00	2023-07-07 00:00:00
5	25	1	48795.00	2023-12-04 00:00:00	2023-12-13 00:00:00
6	33	1	86815.00	2023-02-23 00:00:00	2023-03-17 00:00:00
7	33	1	84615.00	2023-02-23 00:00:00	2023-03-17 00:00:00
8	42	1	34237.00	2023-02-14 00:00:00	2023-03-01 00:00:00
9	43	1	53994.00	2023-04-05 00:00:00	2023-04-25 00:00:00

2-5 查询订单总金额大于平均订单金额的所有订单信息。

```

SELECT
    o.order_id,
    o.user_id,
    u.username,
    u.email,
    o.total_amount,
    o.create_time,
    o.update_time
FROM "订单基本信息表" o
INNER JOIN "用户基本信息表" u ON o.user_id = u.user_id
WHERE o.total_amount > (
    SELECT AVG(total_amount)
    FROM "订单基本信息表"
)
ORDER BY o.total_amount DESC;

```

	order_id	user_id	username	email	total_amount	create_time	update_time
1	4404	45	Eva	evaj@example.com	123576.00	2023-09-04 00:00:00	2023-09-02 00:00:00
2	8795	88	Vera	veraj@example.com	117568.00	2023-05-24 00:00:00	2023-06-13 00:00:00
3	576	6	陈八	chou8@example.com	115384.00	2023-01-28 00:00:00	2023-02-17 00:00:00
4	693	7	吴凡	wujia@example.com	113126.00	2023-07-16 00:00:00	2023-07-23 00:00:00
5	6668	67	Ava	avaj@example.com	111777.00	2023-02-19 00:00:00	2023-03-17 00:00:00
6	2320	24	李娜	linaj@example.com	109967.00	2023-01-29 00:00:00	2023-02-18 00:00:00
7	7856	71	Ellie	elliaj@example.com	103847.00	2023-09-24 00:00:00	2023-09-30 00:00:00
8	6988	70	Duke	dukej@example.com	103623.00	2023-01-20 00:00:00	2023-02-01 00:00:00
9	8532	86	Zack	zack@example.com	101905.00	2023-02-11 00:00:00	2023-03-03 00:00:00

2-6 查询订单总金额大于任意一个用户的最小订单金额的订单信息。

SELECT

o.order_id,

o.user_id,

u.username,

u.email,

o.total_amount,

o.create_time,

o.update_time

FROM "订单基本信息表" o

INNER JOIN "用户基本信息表" u ON o.user_id = u.user_id

WHERE o.total_amount > ANY (

SELECT MIN(total_amount)

FROM "订单基本信息表"

GROUP BY user_id

)

ORDER BY o.total_amount DESC;

	order_id	user_id	username	email	total_amount	create_time	update_time
1	4404	45	Eva	evaj@example.com	123576.00	2023-09-04 00:00:00	2023-09-02 00:00:00
2	8795	88	Vera	veraj@example.com	117568.00	2023-05-24 00:00:00	2023-06-13 00:00:00
3	576	6	陈八	chou8@example.com	115384.00	2023-01-28 00:00:00	2023-02-17 00:00:00
4	693	7	吴凡	wujia@example.com	113126.00	2023-07-16 00:00:00	2023-07-23 00:00:00
5	6668	67	Ava	avaj@example.com	111777.00	2023-02-19 00:00:00	2023-03-17 00:00:00
6	2320	24	李娜	linaj@example.com	109967.00	2023-01-29 00:00:00	2023-02-18 00:00:00
7	7856	71	Ellie	elliaj@example.com	103847.00	2023-09-24 00:00:00	2023-09-30 00:00:00
8	6988	70	Duke	dukej@example.com	103623.00	2023-01-20 00:00:00	2023-02-01 00:00:00
9	8532	86	Zack	zack@example.com	101905.00	2023-02-11 00:00:00	2023-03-03 00:00:00

2-7 查询订单总金额大于所有用户的平均订单金额的所有订单信息。

SELECT

o.order_id,

o.user_id,

u.username,

```

u.email,

o.total_amount,

o.create_time,

o.update_time

FROM "订单基本信息表" o

INNER JOIN "用户基本信息表" u ON o.user_id = u.user_id

WHERE o.total_amount > ALL (

    SELECT AVG(total_amount)

    FROM "订单基本信息表"

    GROUP BY user_id

)

ORDER BY o.total_amount DESC;

```

	order_id	user_id	username	email	total_amount	create_time	update_time
1	8854	45	Eva	evaj@example.com	123576.00	2023-08-04 00:00:00	2023-09-02 00:00:00
2	8795	88	Vera	veraj@example.com	117568.00	2023-05-24 00:00:00	2023-06-13 00:00:00
3	576	6	陈八	chou8@example.com	115384.00	2023-01-28 00:00:00	2023-02-17 00:00:00
4	683	7	吴九	wuj@example.com	113126.00	2023-07-16 00:00:00	2023-07-23 00:00:00
5	6668	67	Ava	avaj@example.com	111777.00	2023-02-19 00:00:00	2023-03-17 00:00:00
6	2328	24	李娜	linaj@example.com	10997.00	2023-01-29 00:00:00	2023-02-18 00:00:00
7	3956	75	Ellie	ellaj@example.com	103847.00	2023-09-24 00:00:00	2023-09-30 00:00:00
8	6988	39	Duke	dukej@example.com	182623.00	2023-01-20 00:00:00	2023-02-01 00:00:00
9	6532	66	Zack	zackd@example.com	381985.00	2023-02-11 00:00:00	2023-03-03 00:00:00

2-8 使用EXISTS谓词查询至少购买过一次商品的用户信息。

```

SELECT

u.user_id,

u.username,

u.email,

u.phone,

u.age,

u.gender,

u.create_time,

u.update_time

FROM "用户基本信息表" u

WHERE EXISTS (

    SELECT *

    FROM "订单基本信息表" o

    WHERE o.user_id = u.user_id

```

)

ORDER BY u.user_id;

	user_id	username	email	phone	age	gender	create_time	update_time
1	1	张三	zhangsan@example.com	13800138000	28	男	2023-01-15 00:00:00	2023-10-01 00:00:00
2	2	李四	lisi@example.com	13800138001	34	男	2023-02-10 00:00:00	2023-10-02 00:00:00
3	3	王五	wangwu@example.com	13800138002	22	男	2023-03-05 00:00:00	2023-10-03 00:00:00
4	4	赵六	zhao6@example.com	13800138003	30	女	2023-04-20 00:00:00	2023-10-04 00:00:00
5	5	孙七	sun7@example.com	13800138004	25	女	2023-05-12 00:00:00	2023-10-05 00:00:00
6	6	周八	zhou8@example.com	13800138005	40	男	2023-06-18 00:00:00	2023-10-06 00:00:00
7	7	吴九	wu9@example.com	13800138006	19	女	2023-07-22 00:00:00	2023-10-07 00:00:00
8	8	郑十	zheng10@example.com	13800138007	27	男	2023-08-30 00:00:00	2023-10-08 00:00:00
9	9	王明	wangming@example.com	13800138008	31	女	2023-09-05 00:00:00	2023-10-09 00:00:00

2-9 查询每个商品分类的名称及其总销售额。

SELECT

c.category_id,

c.name AS category_name,

COALESCE(SUM(oi.quantity * oi.price), 0) AS total_sales

FROM "商品分类基本信息表" c

LEFT JOIN "订单明细基本信息表" oi ON p.product_id = oi.product_id

GROUP BY c.category_id, c.name

ORDER BY total_sales DESC;

	category_id	category_name	total_sales
1	10	笔记本电脑	32345499.00
2	16	手机	29993667.00
3	17	平板电脑	27756500.00
4	21	冰箱	24559491.00
5	20	电视	20200118.00
6	24	沙发	13744063.00
7	25	床	10304564.00
8	17	桌子	2741369.00
9	23	微波炉	2672976.00

3. 集合查询：

3-1 查询所有购买了商品10或商品20的订单（包括重复订单）。

SELECT DISTINCT

o.order_id,

o.user_id,

u.username,

o.total_amount,

o.create_time

FROM "订单基本信息表" o

INNER JOIN "订单明细基本信息表" oi ON o.order_id = oi.order_id

```

INNER JOIN "用户基本信息表" u ON o.user_id = u.user_id

WHERE oi.product_id IN (10, 20)

ORDER BY o.order_id;

```

	order_id	user_id	username	total_amount	create_time
1	1	1	张三	20030.00	2023-09-04 00:00:00
2	2	1	张三	2799.00	2023-05-31 00:00:00
3	8	1	张三	45890.00	2023-05-30 00:00:00
4	15	1	张三	45326.00	2023-07-23 00:00:00
5	20	1	张三	34252.00	2023-05-24 00:00:00
6	23	1	张三	58214.00	2023-06-25 00:00:00
7	25	1	张三	48705.00	2023-12-04 00:00:00
8	33	1	张三	84615.00	2023-02-23 00:00:00
9	38	1	张三	705.00	2023-07-02 00:00:00

3-2 查询所有同时购买了商品10和商品20的订单。

```

SELECT

o.order_id,

o.user_id,

u.username,

o.total_amount,

o.create_time

FROM "订单基本信息表" o

INNER JOIN "用户基本信息表" u ON o.user_id = u.user_id

WHERE EXISTS (

    SELECT *

    FROM "订单明细基本信息表" oi1

    WHERE oi1.order_id = o.order_id AND oi1.product_id = 10

)

AND EXISTS (

    SELECT *

    FROM "订单明细基本信息表" oi2

    WHERE oi2.order_id = o.order_id AND oi2.product_id = 20

)

ORDER BY o.order_id;

```

	order_id	user_id	username	total_amount	create_time
1	33	3	张三	84615.00	2023-02-23 00:00:00
2	63	3	张三	18549.00	2023-06-22 00:00:00
3	347	2	李四	13734.00	2023-01-01 00:00:00
4	390	2	李四	28323.00	2023-04-06 00:00:00
5	292	3	王五	44518.00	2023-06-29 00:00:00
6	358	4	赵六	45511.00	2023-06-29 00:00:00
7	380	4	赵六	19529.00	2023-12-04 00:00:00
8	435	5	孙七	18089.00	2023-11-29 00:00:00
9	652	7	吴九	13233.00	2023-07-22 00:00:00

3-3 查询购买了商品10但没有购买商品20的订单

SELECT

o.order_id,

o.user_id,

u.username,

o.total_amount,

o.create_time

FROM "订单基本信息表" o

INNER JOIN "用户基本信息表" u ON o.user_id = u.user_id

WHERE EXISTS (

SELECT *

FROM "订单明细基本信息表" oi1

WHERE oi1.order_id = o.order_id AND oi1.product_id = 10

)

AND NOT EXISTS (

SELECT *

FROM "订单明细基本信息表" oi2

WHERE oi2.order_id = o.order_id AND oi2.product_id = 20

)

ORDER BY o.order_id;

	order_id	user_id	username	total_amount	create_time
1	8	3	张三	45000.00	2023-01-30 00:00:00
2	35	3	张三	45326.00	2023-07-23 00:00:00
3	23	3	张三	38234.00	2023-06-25 00:00:00
4	25	3	张三	48705.00	2023-12-04 00:00:00
5	42	3	张三	36237.00	2023-02-14 00:00:00
6	43	3	张三	53904.00	2023-04-05 00:00:00
7	33	3	张三	62386.00	2023-12-20 00:00:00
8	59	3	张三	28711.00	2023-06-26 00:00:00
9	64	3	张三	13285.00	2023-10-02 00:00:00

实验小结

通过本次数据查询实验，我深入掌握了SQL语言在PostgreSQL中的实际应用，包括单表查询、多表连接、嵌套查询及集合查询等多种操作方式。在单表查询部分，我练习了基础数据筛选、聚合函数、分组统计、排序及条件筛选等核心语法，并学会了使用CASE语句进行数据分段分析。在多表查询与嵌套查询环节，我熟悉了INNER JOIN、LEFT JOIN等表连接方法，以及EXISTS、ANY、ALL等嵌套查询技巧，提升了处理复杂业务逻辑的能力。此外，集合查询部分让我进一步理解了如何通过联合、交集和差集操作实现多条件数据检索。整体上，本实验有效巩固了我对SQL查询语句的理解与应用能力，为今后进行更复杂的数据分析与数据库操作打下了坚实基础。